

Making Web Applications Agent-Accessible via MCP

A Tradeoff Analysis of REST API and Model Context Protocol
Integration

1. Project Report

at the School of Business
in the study program Data Science & Künstliche Intelligenz

at the
DHBW Ravensburg

Author:	Max Mustermann
Course:	Musterkurs
DHBW Supervisor:	Prof. Dr.-Ing. Mark Schutera
Dual Partner:	Musterfirma
Submission Date:	Musterdatum

Contents

List of Abbreviations II

List of Tables III

1 Introduction 1

1.1 Problem Statement 1

1.2 Objectives 1

2 Methods 2

2.1 MCP Server Design 2

2.2 Evaluation Methodology 2

3 Implementation Challenges 3

4 Results 4

4.1 Latency 4

4.2 Integration Complexity 4

5 Discussion 5

5.1 Limitations and Future Work 5

5.2 Conclusion 5

Appendix A: AI Usage Declaration 6

Bibliography 7

List of Abbreviations

AI	Artificial Intelligence
API	Application Programming Interface
AWI	Agentic Web Interface
HTTP	Hypertext Transfer Protocol
LLM	Large Language Model
MCP	Model Context Protocol
REST	Representational State Transfer
VM	Virtual Machine

List of Tables

2.1	MCP tools	2
4.1	Latency comparison	4
4.2	Complexity comparison	4

1 Introduction

Web applications are designed for human interaction through graphical interfaces. Artificial Intelligence (AI) agents that attempt to use these interfaces through browser automation face low reliability: on the WebArena benchmark, hybrid agents combining Application Programming Interface (API) access with browsing outperform browser-only agents by over 24 percentage points (Song et al., 2024, p. 1). This gap reflects a fundamental mismatch: current web interfaces are designed for human users, not for autonomous agents (Lù et al., 2025, p. 1).

This work addresses this mismatch by extending an existing full-stack web application—PixelWise, a digit classification system deployed on Virtual Machine (VM)s as part of the DHBW “Full Stack Handwerk” course (Schutera, 2026, p. 75) (Block 5: Backend APIs & Services)—with a standardized agent interface using the Model Context Protocol (MCP) (Anthropic, 2024). The extension does not modify the existing application stack; it adds a separate interface layer that wraps the existing Representational State Transfer (REST) API.

1.1 Problem Statement

Modern web applications already expose internal APIs behind their user interfaces. However, these APIs lack a standardized discovery mechanism: an agent must know endpoint paths, Hypertext Transfer Protocol (HTTP) methods, authentication headers, and request body formats before it can make a single call. No machine-readable description tells the agent what operations are available or how to invoke them.

MCP addresses this gap by providing a protocol with built-in tool discovery (`tools/list`) and structured invocation (`tools/call`) via JSON-RPC 2.0 (Anthropic, 2024). As of December 2025, the protocol has been donated to the Linux Foundation’s Agentic AI Foundation, with over 10,000 active servers and 97 million monthly SDK downloads (Anthropic, 2025).

1.2 Objectives

This project makes the following contributions:

- An MCP server that exposes the PixelWise digit classifier to AI agents through three structured tools (`check_health`, `classify_digit`, `get_prediction_history`).

- A quantitative comparison of direct REST API usage versus MCP-mediated access across two dimensions: latency and integration complexity.
- Reproducible benchmark scripts that measure invocation success rate, task success rate, and end-to-end latency without requiring an Large Language Model (LLM) or paid API key.

2 Methods

The extension adds an MCP server as a standalone Python process on the client machine that forwards agent requests to the existing PixelWise REST API on the production VM. It supports two transports: `stdio` (agent spawns server as subprocess) and `Streamable HTTP` (server runs as standalone service on port 3000). This design leaves the deployed application stack entirely unchanged.

2.1 MCP Server Design

The server is implemented using the official MCP Python SDK (v1.28.1) with the `FastMCP` high-level API and `httpx` (v0.28.1) for asynchronous HTTP calls. It exposes three tools that map one-to-one to the existing REST endpoints (see Table 2.1).

Tab. 2.1: MCP tools exposed by the server, with their corresponding REST endpoints.

MCP Tool	REST Endpoint	Method	Auth
<code>check_health</code>	<code>/health</code>	GET	None
<code>classify_digit</code>	<code>/classify</code>	POST	API Key
<code>get_prediction_history</code>	<code>/results</code>	GET	None

Each tool description specifies its purpose, input format, and return value. For `classify_digit`, the description explains that the input is a 28×28 pixel grid with values 0–255 where pixels above 128 represent ink, and the classifier recognizes digits 1 through 9. The API key is passed via environment variable, not hardcoded. When the REST API returns an error (401, 500), the MCP server propagates it as a structured MCP error response rather than crashing.

2.2 Evaluation Methodology

We compare REST and MCP along two dimensions, supplemented by MCP-specific quality metrics.

Latency. A Python benchmark script sends $N = 30$ classification requests—cycling through 10 distinct test digit patterns (0–9)—and records per-request latency using `time.perf_counter()`. Both REST and MCP benchmarks use asynchronous `httpx` clients, include a warm-up request excluded from measurement, and respect the API rate limit of 30 requests per minute via 2.1 s inter-request delays. We report mean ($\pm$ standard deviation), median, and 95th percentile latency.

Integration complexity. We define three proxies to quantify the effort required to accomplish the same task via REST versus MCP: (a) lines of client code, (b) number of hardcoded values (URLs, header names, JSON keys), and (c) number of technical concepts a developer must understand. These are not standardized metrics; we use them as practical operationalizations of integration effort and report them transparently.

MCP quality. Inspired by the rule-based evaluation of MCP-Bench (Wang et al., 2025, pp. 7–8), we measure tool invocation success rate (fraction of `tools/call` requests that return without error) and task success rate (fraction of correctly delivered classification results). Edge-case tests verify error handling for invalid API keys, malformed pixel data, and empty inputs.

All benchmark scripts run without an LLM or paid AI service key. The MCP server is spawned as a subprocess by the benchmark script itself, using the MCP SDK’s client library. Measurements were taken against a PixelWise production instance on a UTM VM, accessed from a MacBook Air M1 via shared network. Source code and instructions are provided in the project repository.

3 Implementation Challenges

Rate limiting. The API rate limit (30 requests/minute) caused HTTP 429 errors during initial benchmark runs, inflating error counts and distorting latency measurements. We added 2.1 s inter-request delays and 60 s cooldowns between measurement phases, which also informed the choice of $N = 30$ iterations.

Transport selection. We started with stdio transport (required by Claude Desktop and Claude Code) and later added Streamable HTTP for standalone testing via `curl` without requiring an AI client. Both are selectable via a `--transport` flag.

Benchmark validity. Initial benchmarks used a single test digit for all iterations, making 100% success trivially expected. We replaced this with 10 rotating digit patterns (0–9) to test the full pipeline with varied inputs while keeping the benchmark deterministic.

4 Results

4.1 Latency

Table 4.1 summarizes the latency measurements over 30 iterations with 10 rotating test digit patterns.

Tab. 4.1: Latency comparison between direct REST API calls and MCP tool calls ($N = 30$, 10 digit patterns, warm-up excluded).

Metric	REST API [ms]	MCP Server [ms]
Mean $\pm$ SD	15.63 ± 7.06	41.62 ± 7.19
Median	13.85	41.23
95th percentile	37.26	54.43
Errors	0/30	0/30

The MCP server adds a mean overhead of 26 ms, which corresponds to the JSON-RPC framing, tool dispatch, and subprocess communication layers. Both approaches achieve zero errors across all 30 iterations. REST latency is right-skewed (mean 15.63 ms vs. median 13.85 ms, with two outliers above 35 ms attributed to VM scheduling jitter), while MCP latency is symmetrically distributed around the median.

4.2 Integration Complexity

Table 4.2 compares the client-side effort for two tasks of increasing complexity.

Tab. 4.2: Integration complexity for two tasks. Task A: classify a single digit. Task B: health check, classify, retrieve history.

Metric	REST		MCP	
	Task A	Task B	Task A	Task B
Lines of code	8	22	0	0
Hardcoded values	4	10	0	0
Required concepts	5	7	1	1

With REST, the agent must know endpoint paths, HTTP methods, the `X-API-Key` header name, and the JSON body structure—all hardcoded into client code. From Task A to Task B, lines of code increase by 175% and hardcoded values by 150%. With MCP, the agent calls `tools/list` to discover available tools, then invokes them with structured arguments. Complexity remains constant regardless of task size.

Tool discovery returned all three tools with complete schemas; invocation and task success rates were both 3/3 (100%). Edge-case tests confirmed correct error propagation: invalid

API key (401), wrong pixel dimensions (500), and empty input were all returned as structured MCP errors (`isError: true`) rather than crashes.

5 Discussion

Direct REST access is faster by 26 ms per call, but MCP eliminates all client-side code through automatic tool discovery. This tradeoff echoes a broader pattern: Song et al. show that hybrid agents combining API access with browsing outperform browser-only agents by over 24 percentage points on WebArena (Song et al., 2024, p. 7). The 166% relative overhead is negligible in absolute terms: 42 ms is insignificant compared to LLM inference times of 500 ms to several seconds.

MCP’s decisive advantage is constant complexity: zero client code regardless of task size. This aligns with Lù et al.’s standardization principle for Agentic Web Interface (AWI)s (Lù et al., 2025, p. 5)—any agent can discover and invoke tools without application-specific code. The current implementation runs reliably against the production VM with both transports, achieving 100% success rates across all benchmarks. Testing was limited to macOS (Apple Silicon); Linux compatibility is expected but unverified.

5.1 Limitations and Future Work

The complexity metrics (lines of code, hardcoded values, required concepts) are our own operationalization, not standardized measures; we report them transparently as practical proxies for integration effort. The evaluation covers a single application with three tools ($N = 30$ iterations), limiting both generalizability and tail-percentile precision. Two extensions follow naturally: trajectory-level and Pass^k evaluation (Yao et al., 2024, pp. 4–5) with live LLM agents would test whether tool descriptions suffice for autonomous multi-step completion, and applying the MCP pattern to larger APIs would test scalability.

5.2 Conclusion

This work extended PixelWise with an MCP server exposing three tools via stdio and Streamable HTTP transport, achieving 100% success rates across all benchmarks (Section 1.2). The quantitative comparison shows 26 ms mean latency overhead while eliminating all client-side code (8–22 lines to zero) and hardcoded values (4–10 to zero). All benchmark scripts run without an LLM or paid API key and are available in the project repository for independent reproduction.

Appendix A: AI Usage Declaration

The following table documents all uses of AI tools beyond the whitelisted categories (spell checking, grammar, thesaurus, translation, proofreading, code error checking) as required by DHBW guidelines.

AI Tool	Prompts / Usage	Purpose	Sections
Claude Code (Anthropic, Opus 4.6)	Conversational exploration of project scope, evaluation dimensions, and report structure	Developing understanding of what to build, defining the comparison framework (latency, complexity, MCP quality), and outlining the report	All sections
Claude Code (Anthropic, Opus 4.6)	Iterative conversational coding of the MCP server	Implementation of MCP server with three tools, stdio and Streamable HTTP transport	Section 2
Claude Code (Anthropic, Opus 4.6)	Iterative conversational coding of benchmark and evaluation scripts	Implementation of latency benchmark, Streamable HTTP benchmark, edge-case tests, and REST client baseline	Sections 2, 4
Claude Code (Anthropic, Opus 4.6)	Collaborative creation of project documentation	Writing the README with setup instructions, architecture description, and reproduction steps	Project repository
Claude Code (Anthropic, Opus 4.6)	Assistance with text formulation based on author-provided content direction and documented technical results	Reformulating and drafting report text; author reviewed and approved all output	All sections

Bibliography

- Anthropic. (2024). Model context protocol specification (2025-11-25) [Accessed: 2026-06-27]. <https://modelcontextprotocol.io/specification/2025-11-25>
- Anthropic. (2025). Donating the model context protocol and establishing the agentic AI foundation [Accessed: 2026-06-30]. <https://www.anthropic.com/news/donating-the-model-context-protocol-and-establishing-of-the-agentic-ai-foundation>
- Lù, X. H., Kamath, G., Mosbach, M., & Reddy, S. (2025). Build the web for agents, not agents for the web. *arXiv preprint*. <https://arxiv.org/abs/2506.10953>
- Schutera, M. (2026). Full stack handwerk [Unfinished lecture notes, DHBW Ravensburg]. https://github.com/Quillstacks/LectureMaterial/tree/main/lecturenotes/notes_missingsemester
- Song, Y., Xu, F., Zhou, S., & Neubig, G. (2024). Beyond browsing: API-based web agents. *arXiv preprint*. <https://arxiv.org/abs/2410.16464>
- Wang, Z., et al. (2025). MCP-Bench: Benchmarking tool-using LLM agents with complex real-world tasks via MCP servers. *arXiv preprint*. <https://arxiv.org/abs/2508.20453>
- Yao, S., Shinn, N., Razavi, P., & Narasimhan, K. (2024). τ -bench: A benchmark for tool-agent-user interaction in real-world domains. *arXiv preprint*. <https://arxiv.org/abs/2406.12045>

Selbständigkeitserklärung

Ich versichere hiermit, dass ich die vorliegende Arbeit selbstständig verfasst und keine anderen als die angegebenen Quellen und Hilfsmittel verwendet habe und diese Arbeit bei keiner anderen Prüfung mit gleichem oder vergleichbarem Inhalt vorgelegt habe und diese bislang nicht veröffentlicht wurde. Des Weiteren versichere ich, dass die eingereichte elektronische Fassung mit der gedruckten Ausfertigung übereinstimmt.

Musterort, Musterdatum

Ort, Datum

Unterschrift